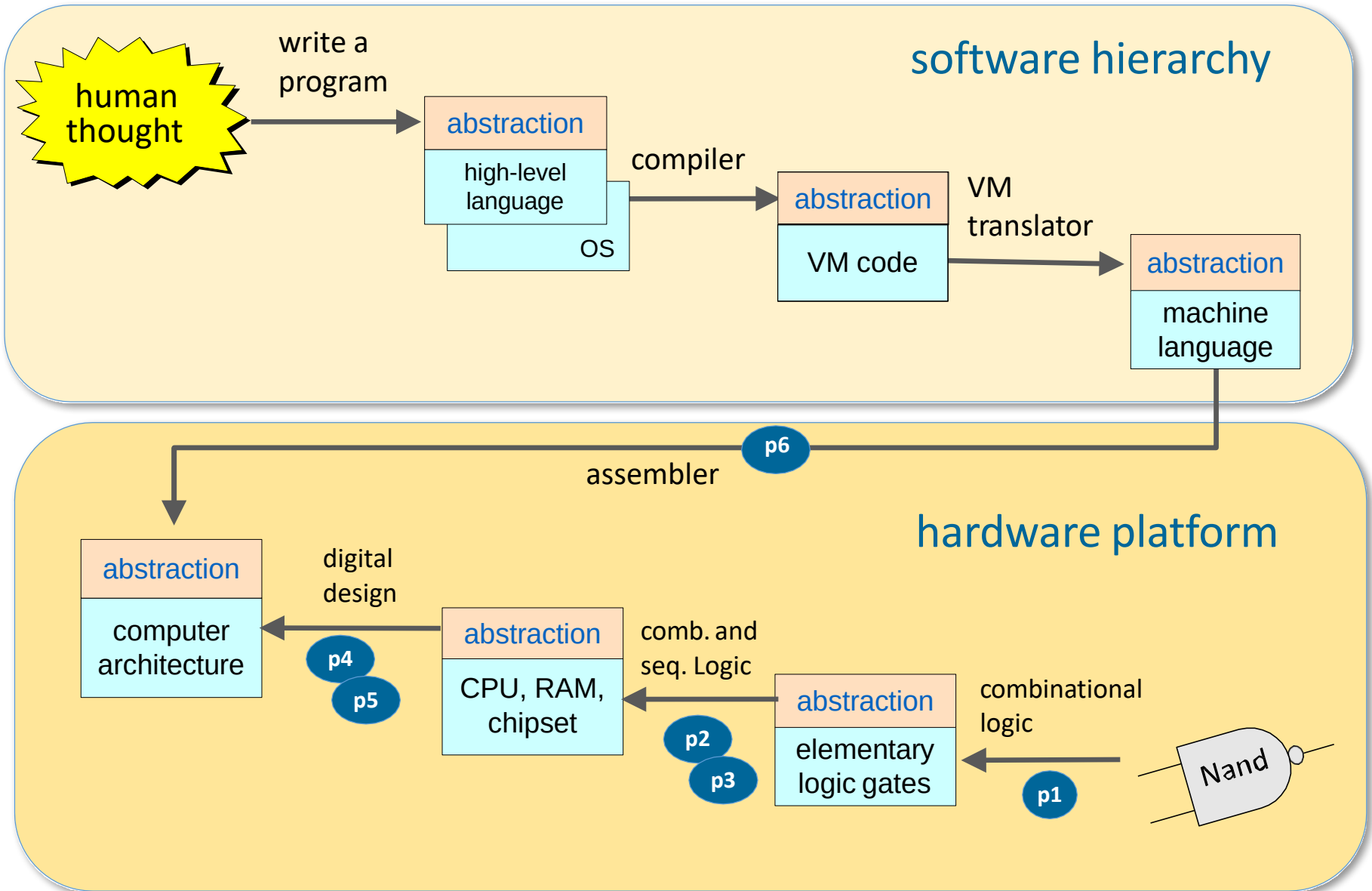




22AIE113 Elements of Computing -2 Assembler

Department of Computer Science & Engineering,
Amrita School of Engineering
Amrita Vishwa Vidyapeetham

Nand to Tetris: the big picture



Assembly process

Assembly Language

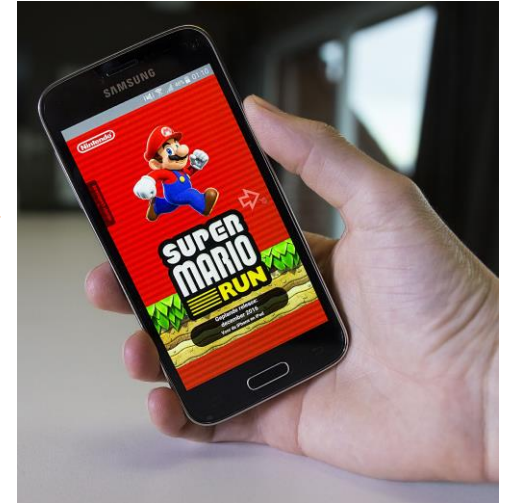
```
@i
M=1 // i = 1
@sum
M=0 // sum = 0
(LOOP)
@i // if i>RAM[0]
D=M // GOTO WRITE
@R0
D=D-M
@WRITE
D;JGT
... // Etc.
```

assembler

Machine Language

```
00000000000010000
1110111111001000
00000000000010001
1110101010001000
00000000000010000
11111100000010000
00000000000000000
1111010011010000
00000000000010010
11100011000000001
00000000000010000
11111100000010000
00000000000010001
...
```

run



Assembler: lecture plan



The assembly process



The Hack assembly language

- The assembly process: instructions
- The assembly process: symbols
- Developing an assembler
- Project 6 overview

The translator's challenge (overview)

Hack assembly code (source language)

```
// Computes RAM[1]=1+...+RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i      // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i      // sum += i
D=M
@sum
M=D+M
@i      // i++
M=M+1
@LOOP  // goto LOOP
0;JMP
...
```

Assembler

Hack binary code (source language)

```
00000000000010000
1110111111001000
00000000000010001
1110101010001000
00000000000010000
11111100000010000
00000000000000000
1111010011010000
00000000000010010
11100011100000001
00000000000010000
11111100000010000
00000000000010001
1111000010001000
00000000000010000
1111110111001000
00000000000000100
1110101010000111
...
```

Based on the syntax
rules of:

- The source language
- The target language

Hack language specification: A-instruction

Symbolic syntax:

`@value`

Where *value* is either

- a non-negative decimal constant or
- a symbol referring to such a constant

Binary syntax:

`0valueInBinary`

Examples:

`@21`

`@foo`

Example:

`00000000000010101`

Reference: <http://nand2tetris.org>

Hack language specification: C-instruction

Symbolic syntax: *dest* = *comp* ; *jump*

Binary syntax: 1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

<i>comp</i>		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D A	D M	0	1	0	1	0	1
a=0	a=1						

<i>dest</i>	d1	d2	d3	effect: the value is stored in:
null	0	0	0	The value is not stored
M	0	0	1	RAM[A]
D	0	1	0	D register
MD	0	1	1	RAM[A] and D register
A	1	0	0	A register
AM	1	0	1	A register and RAM[A]
AD	1	1	0	A register and D register
AMD	1	1	1	A register, RAM[A], and D register

<i>jump</i>	j1	j2	j3	effect:
null	0	0	0	no jump
JGT	0	0	1	if out > 0 jump
JEQ	0	1	0	if out = 0 jump
JGE	0	1	1	if out ≥ 0 jump
JLT	1	0	0	if out < 0 jump
JNE	1	0	1	if out ≠ 0 jump
JLE	1	1	0	if out ≤ 0 jump
JMP	1	1	1	Unconditional jump

Hack language specification: symbols

Pre-defined symbols:

<u>symbol</u>	<u>value</u>	<u>symbol</u>	<u>value</u>
R0	0	SP	0
R1	1	LCL	1
R2	2	ARG	2
...	...	THIS	3
R15	15	THAT	4
SCREEN	16384		
KBD	24576		

Label declaration: (*label*)

Variable declaration: *@variableName*

The Hack language: a translator's perspective

Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i      // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i      // sum += i
D=M
@sum
M=D+M
@i      // i++
M=M+1
@LLOOP // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

Assembly program elements:

- White space
 - ❑ Empty lines / indentation
 - ❑ Line comments
 - ❑ In-line comments
- Instructions
 - ❑ A-instructions
 - ❑ C-instructions
- Symbols
 - ❑ References
 - ❑ Label declarations

The Hack language: a translator's perspective

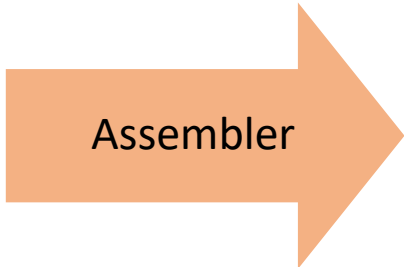
Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i      // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i      // sum += i
D=M
@sum
M=D+M
@i      // i++
M=M+1
@LLOOP // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```



Assembler

Challenges:

Handling...

- White space
- Instructions
- Symbols

Hack machine code

```
00000000000010000
1110111111001000
00000000000010001
1110101010001000
00000000000010000
11111100000010000
00000000000000000
1111010011010000
00000000000010010
11100011000000001
00000000000010000
11111100000010000
00000000000010001
1111000010001000
00000000000010000
1111110111001000
00000000000000100
1110101010000111
00000000000010001
11111100000010000
00000000000000001
1110001100001000
00000000000010110
1110101010000111
```

Symbols

Program with symbols

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i     // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i     // sum += i
D=M
@sum
M=D+M
@i     // i++
M=M+1
@LOOP  // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

Challenges:

Handling...

- White space
- Instructions
- Symbols

Simplifying assumption:

Let's deal with symbols later.

Handling programs without symbols

Assembly program (without symbols)

```
// Computes RAM[1] = 1 + ... + RAM[0]
@16
M=1    // i = 1
@17
M=0    // sum = 0

@16    // if i>RAM[0] goto STOP
D=M
@0
D=D-M
@18
D;JGT
@16    // sum += i
D=M
@17
M=D+M
@16    // i++
M=M+1
@4     // goto LOOP
0;JMP
@17
D=M
@1
M=D
@22    // RAM[1] = the sum
0;JMP
```

Assembler
for symbol-less
Hack programs

Challenges:

Handling...

- White space
- Instructions

Hack machine code

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
00000000000000100
1110101010000111
0000000000010001
1111110000010000
00000000000000001
1110001100001000
0000000000010110
1110101010000111
```



Handling white space

Assembly program (without symbols)

```
// Computes RAM[1] = 1 + ... + RAM[0]
@16
M=1    // i = 1
@17
M=0    // sum = 0

@16    // if i>RAM[0] goto STOP
D=M
@0
D=D-M
@18
D;JGT
@16    // sum += i
D=M
@17
M=D+M
@16    // i++
M=M+1
@4     // goto LOOP
0;JMP
@17
D=M
@1
M=D    // RAM[1] = the sum
@22
0;JMP
```

Assembler
for symbol-less
Hack programs

Challenges:

Handling...

- White space
- Instructions

Handling white space:

Ignore it!

Hack machine code

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000000100
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```



Handling instructions

Assembly program (without symbols)

```
@16
M=1
@17
M=0
@16
D=M
@0
D=D-M
@18
D;JGT
@16
D=M
@17
M=D+M
@16
M=M+1
@4
0;JMP
@17
D=M
@1
M=D
@22
0;JMP
```

Assembler
for symbol-less
Hack programs

Challenges:

Handling...

- ✓ White space
- Instructions

Hack machine code

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000000100
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```

Translating A-instructions

Symbolic syntax:

@value

Examples:

@21

@foo

Where *value* is either

- a non-negative decimal constant or
- a symbol referring to such a constant (later)

Binary syntax:

0 valueInBinary

Example:

00000000000010101

Translation to binary:

- If *value* is a decimal constant, generate the equivalent binary constant
- If *value* is a symbol, later.

Translating C-instructions

Symbolic syntax:

dest = *comp* ; *jump*

Binary syntax:

1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

<i>comp</i>		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D A	D M	0	1	0	1	0	1
a=0	a=1						

<i>dest</i>	d1	d2	d3	effect: the value is stored in:
null	0	0	0	The value is not stored
M	0	0	1	RAM[A]
D	0	1	0	D register
MD	0	1	1	RAM[A] and D register
A	1	0	0	A register
AM	1	0	1	A register and RAM[A]
AD	1	1	0	A register and D register
AMD	1	1	1	A register, RAM[A], and D register

<i>jump</i>	j1	j2	j3	effect:
null	0	0	0	no jump
JGT	0	0	1	if out > 0 jump
JEQ	0	1	0	if out = 0 jump
JGE	0	1	1	if out ≥ 0 jump
JLT	1	0	0	if out < 0 jump
JNE	1	0	1	if out ≠ 0 jump
JLE	1	1	0	if out ≤ 0 jump
JMP	1	1	1	Unconditional jump

Symbolic:

Binary:

Example:

MD=D+1



Translating C-instructions

Symbolic syntax:

dest = *comp* ; *jump*

Binary syntax:

1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

<i>comp</i>		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D A	D M	0	1	0	1	0	1
a=0	a=1						

<i>dest</i>	d1	d2	d3	effect: the value is stored in:
null	0	0	0	The value is not stored
M	0	0	1	RAM[A]
D	0	1	0	D register
MD	0	1	1	RAM[A] and D register
A	1	0	0	A register
AM	1	0	1	A register and RAM[A]
AD	1	1	0	A register and D register
AMD	1	1	1	A register, RAM[A], and D register

<i>jump</i>	j1	j2	j3	effect:
null	0	0	0	no jump
JGT	0	0	1	if out > 0 jump
JEQ	0	1	0	if out = 0 jump
JGE	0	1	1	if out ≥ 0 jump
JLT	1	0	0	if out < 0 jump
JNE	1	0	1	if out ≠ 0 jump
JLE	1	1	0	if out ≤ 0 jump
JMP	1	1	1	Unconditional jump

Symbolic:

Binary:

Example:

MD=D+1

1110011111011000

The overall assembly logic

Assembly program

```
@16
M=1
@17
M=0
@16
D=M
@0
D=D-M
@18
D;JGT
@16
D=M
@17
M=D+M
@16
M=M+1
@4
0;JMP
@17
D=M
@1
M=D
@22
0;JMP
```

For each instruction

- Parse the instruction:
break it into its underlying fields
- A-instruction:
translate the decimal value into a binary value
- C-instruction:
for each field in the instruction, generate the
corresponding binary code;
- Assemble the translated binary codes into a complete
16-bit machine instruction
- Write the 16-bit instruction to the output file.

The overall assembly logic

Assembly program

```
@16
M=1
@17
M=0
@16
D=M
@0
D=D-M
@18
D; JGT
@16
D=M
@17
M=D+M
@16
M=M+1
@4
0; JMP
@17
D=M
@1
M=D
@22
0; JMP
```

Resulting code:

Disclaimer

The source code
contains no symbols

Hack machine code

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000000100
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```

Assembler: lecture plan



Assembler logic (basic)



The Hack assembly language



The assembly process: instructions



The assembly process: symbols

- Developing an assembler
- Project 6 overview

Handling symbols

Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i     // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i     // sum += i
D=M
@sum
M=D+M
@i     // i++
M=M+1
@LOOP  // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

Pre-defined symbols:

represent special memory locations

label symbols:

represent destinations of
goto instructions

variable symbols:

represent memory locations where the
programmer wants to maintain values

Handling pre-defined symbols

Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LOOP)
@i      // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i      // sum += i
D=M
@sum
M=D+M
@i      // i++
M=M+1
@LOOP  // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

The Hack language specification describes 23 *pre-defined symbols*:

<u>symbol</u>	<u>value</u>	<u>symbol</u>	<u>value</u>
R0	0	SP	0
R1	1	LCL	1
R2	2	ARG	2
...	...	THIS	3
R15	15	THAT	4
SCREEN	16384		
KBD	24576		

Translating @preDefinedSymbol :

Replace *preDefinedSymbol* with its value.

Handling symbols that denote labels

Assembly program

```
0 // Computes RAM[1] = 1 + ... + RAM[0]
1 @i
2 M=1 // i = 1
3 @sum
4 M=0 // sum = 0
5
6 (LOOP)
7 @i // if i>RAM[0] goto STOP
8 D=M
9 @R0
10 D=D-M
11 @STOP
12 D;JGT
13 @i // sum += i
14 D=M
15 @sum
16 M=D+M
17 @i // i++
18 M=M+1
19 @LOOP // goto LOOP
20 0;JMP
21
22 (STOP)
23 @sum
24 D=M
25 @R1
26 M=D // RAM[1] = the sum
27
28 (END)
29 @END
30 0;JMP
```

Label symbols

- Used to label destinations of goto commands
- Declared by the pseudo-command (xxx)
- This directive defines the symbol xxx to refer to the memory location holding the next instruction in the program

<u>symbol</u>	<u>value</u>
LOOP	4
STOP	18
END	22

Translating @labelSymbol :

Replace *labelSymbol* with its value

Handling symbols that denote variables

Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LLOOP)
@i    // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i    // sum += i
D=M
@sum
M=D+M
@i    // i++
M=M+1

@LLOOP // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

Variable symbols

- Any symbol xxx appearing in an assembly program which is not pre-defined and is not defined elsewhere using the (xxx) directive is treated as a *variable*
- Each variable is assigned a unique memory address, starting at 16

<u>symbol</u>	<u>value</u>
i	16
sum	17

Translating @variableSymbol :

- If seen for the first time, assign a unique memory address
- Replace *variableSymbol* with this address

Symbol table

Assembly program

```
0  // Computes RAM[1] = 1 + ... + RAM[0]
1  @i
2  M=1  // i = 1
3  @sum
4  M=0  // sum = 0
5
6  (LOOP)
7  @i  // if i>RAM[0] goto STOP
8  D=M
9  @R0
10 D=D-M
11 @STOP
12 D;JGT
13 @i  // sum += i
14 D=M
15 @sum
16 M=D+M
17 @i  // i++
18 M=M+1
19 @LOOP // goto LOOP
20 0;JMP
21
22 (STOP)
23 @sum
24 D=M
25 @R1
26 M=D  // RAM[1] = the sum
27
28 (END)
29 @END
30 0;JMP
```

Symbol table

symbol	value
R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4
LOOP	4
STOP	18
END	22

Initialization:

Add the pre-defined symbols

First pass:

Add the label symbols

Symbol table

Assembly program

```
// Computes RAM[1] = 1 + ... + RAM[0]
@i
M=1    // i = 1
@sum
M=0    // sum = 0

(LOOP)
@i     // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i     // sum += i
D=M
@sum
M=D+M
@i     // i++
M=M+1
@LOOP  // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D    // RAM[1] = the sum

(END)
@END
0;JMP
```

Symbol table

symbol	value
R0	0
R2	2
R15	15
KBD	24576
LCL	1
THIS	3
LOOP	4
END	22
sum	17

Initialization:

Add the pre-defined symbols

First pass:

Add the label symbols

Second pass:

Add the var. symbols

Usage:

To resolve a symbol, look up its value in the symbol table

The assembly process

Initialization:

- ❑ Construct an empty symbol table
- ❑ Add the pre-defined symbols to the symbol table

First pass:

Scan the entire program;

For each “instruction” of the form (xxx):

- ❑ Add the pair (xxx, *address*) to the symbol table,
where *address* is the number of the instruction following (xxx)

Second pass:

Set *n* to 16

Scan the entire program again; for each instruction:

- ❑ If the instruction is @*symbol*, look up *symbol* in the symbol table;
 - If (*symbol*, *value*) is found, use *value* to complete the instruction’s translation;
 - If not found:
 - Add (*symbol*, *n*) to the symbol table,
 - Use *n* to complete the instruction’s translation,
 - *n*++
- ❑ If the instruction is a C-instruction, complete the instruction’s translation
- ❑ Write the translated instruction to the output file.

Assembler: lecture plan



Assembler logic (basic)



The Hack assembly language



The assembly process: instructions



The assembly process: symbols



Developing an assembler

- Project 6 overview

Sub-tasks that need to be done

- Reading and parsing commands
- Converting mnemonics → code
- Handling symbols

Reading and Parsing Commands

- Start reading a file with a given name
 - E.g. Constructor for a **Parser** object that accepts a string specifying a file name.
 - Need to know how to read text files

Reading and Parsing Commands

- Start reading a file with a given name
- Move to the next command in the file
 - ❑ Are we finished? `boolean hasMoreCommands()`
 - ❑ Get the next command: `void advance()`
 - ❑ Need to read one line at a time
 - ❑ Need to skip whitespace including comments

Reading and Parsing Commands

- Start reading a file with a given name
- Move to the next command in the file
- Get the fields of the current command
 - Type of current command (A-Command, C-Command, or Label)
 - Easy access to the fields:

D=M+1; JGT

@sum



String dest(); String comp(); String jump();

String label();

Translating Mnemonic to Code: overview

Symbolic syntax:

dest = *comp* ; *jump*

Binary syntax:

1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

Translating Mnemonic to Code: computation, destination, jump

Symbolic syntax:

dest = *comp* ; *jump*

Binary syntax:

1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

<i>comp</i>		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D A	D M	0	1	0	1	0	1
a=0	a=1						

<i>jump</i>	j1	j2	j3
null	0	0	0
JGT	0	0	1
JEQ	0	1	0
JGE	0	1	1
JLT	1	0	0
JNE	1	0	1
JLE	1	1	0
JMP	1	1	1

<i>dest</i>	d1	d2	d3
null	0	0	0
M	0	0	1
D	0	1	0
MD	0	1	1
A	1	0	0
AM	1	0	1
AD	1	1	0
AMD	1	1	1

Recap: Parsing + Translating

Symbolic syntax:

dest = *comp* ; *jump*

Binary syntax:

1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

```
// Assume that current command is  
//      D = M+1; JGT
```

```
String c=parser.comp(); // "M+1"  
String d=parser.dest(); // "D"  
String j=parser.jump(); // "JGT"
```

```
String cc = Code.comp(c); // "1110111"  
String dd = Code.dest(d); // "010"  
String jj = Code.jump(j); // "001"
```

```
String out = "111" + cc + dd + jj;
```

The Symbol Table

Symbol	Address
loop	73
sum	12

- Create an new empty table
- Add a (*symbol* , *address*) pair to the table
- Does the table contain a given symbol?
- What is the address associated with a given symbol?

Using the Symbol Table

- Create a new empty table
- Add all the pre-defined symbols to the table
- While reading the input, add labels and new variables to the table
- Whenever you see a “@xxx” command, where xxx is not a number, consult the table to replace the symbol xxx with its address.

Using the Symbol Table: adding symbols

- ...
- ...
- While reading the input, add labels and new variables to the table
 - ❑ **Labels:** when you see a “(xxx)” command, add the symbol xxx and the address of the next machine language command
 - Comment 1: this requires maintaining this running address
 - Comment 2: this may need to be done in a first pass
 - ❑ **Variables:** when you see an “@xxx” command, where xxx is not a number and not already in the table, add the symbol xxx and the next free address for variable allocation

Overall logic

- Initialization
 - Of Parser
 - Of Symbol Table
- First Pass: Read all commands, only paying attention to labels and updating the symbol table
- Restart reading and translating commands
- Main Loop:
 - Get the next Assembly Language Command and parse it
 - For A-commands: Translate symbols to binary addresses
 - For C-commands: get code for each part and put them together
 - Output the resulting machine language command

Developing a Hack Assembler

Contract

- Develop an *assembler* that translates Hack assembly programs into executable Hack binary code
- The source program is supplied in a text file named `xxx.asm`
- The generated code is written into a text file named `xxx.hack`
- Assumption: `xxx.asm` is error-free

Usage

```
prompt> java HackAssembler Xxx.asm
```

This command should create a new `xxx.hack` file that can be executed as-is on the Hack computer.

Parser module: proposed API

Routine	Arguments	Returns	Function
Constructor / initializer	Input file or stream	—	Opens the input file/stream and gets ready to parse it.
hasMoreCommands	—	boolean	Are there more lines in the input?
advance	—	—	- Reads the next command from the input, and makes it the current command. - Takes care of whitespace, if necessary. - Should be called only if <code>hasMoreCommands()</code> is true. - Initially there is no current command.
commandType	—	A_COMMAND, C_COMMAND, L_COMMAND	Returns the type of the current command: A_COMMAND for <code>@xxx</code> where <code>xxx</code> is either a symbol or a decimal number C_COMMAND for <code>dest = comp ; jump</code> L_COMMAND for <code>(xxx)</code> where <code>xxx</code> is a symbol.
symbol	—	string	- Returns the symbol or decimal <code>xxx</code> of the current command <code>@xxx</code> or <code>(xxx)</code>. - Should be called only when <code>commandType()</code> is A_COMMAND or L_COMMAND.
dest	—	string	- Returns the <i>dest</i> mnemonic in the current C-command (8 possibilities). - Should be called only when <code>commandType()</code> is C_COMMAND.
comp	—	string	- Returns the <i>comp</i> mnemonic in the current C-command (28 possibilities). - Should be called only when <code>commandType()</code> is C_COMMAND.
jump	—	string	- Returns the jump mnemonic in the current C-command (8 possibilities). - Should be called only when <code>commandType()</code> is C_COMMAND.

□

Code module: proposed API

Routine	Arguments	Returns	Function
<code>dest</code>	<code>mnemonic (string)</code>	3 bits	Returns the binary code of the <i>dest</i> mnemonic.
<code>comp</code>	<code>mnemonic (string)</code>	7 bits	Returns the binary code of the <i>comp</i> mnemonic.
<code>jump</code>	<code>mnemonic (string)</code>	3 bits	Returns the binary code of the <i>jump</i> mnemonic.

SymbolTable module: proposed API

Routine	Arguments	Returns	Function
Constructor	—	—	Creates a new empty symbol table.
addEntry	symbol (string), address (int)	—	Adds the pair (symbol, address) to the table.
contains	symbol (string)	boolean	Does the symbol table contain the given symbol?
getAddress	symbol (string)	integer	Returns the address associated with the symbol.

Proposed design

The assembler can be implemented in high-level language like Java

Proposed software design

- ❑ Parser: unpacks each instruction into its underlying fields
- ❑ Code: translates each field into its corresponding binary value
- ❑ SymbolTable: manages the symbol table
- ❑ Main: initializes I/O files and drives the process.

Proposed Implementation

Staged development

Assignment Question 1

- Step 1 - First develop a parser program that takes in input .asm file and eliminates the white spaces and comments and generates an output .asm file with purely A&C commands
- Step 2 - Take the above generated output file and identify labels and variables
- Step 3 - Implement a Symbol Table module by combining the above
- Step 4 - Convert a standalone A instruction into Binary
- Step 5 - Convert a standalone C instruction into Binary

Proposed Implementation

Staged development

Assignment Question 2

- Develop the complete assembler that can translate assembly programs with/without symbols
- Extra credit will be given to those who develop the Assembler which can handle symbols

Supplied test programs

Add.asm

Max.asm

MaxL.asm

Rectangle.asm

RectangleL.asm

Test program: Add

Add.asm

```
// Computes RAM[0] = 2 + 3
```

```
@2
```

```
D=A
```

```
@3
```

```
D=D+A
```

```
@0
```

```
M=D
```

Basic test of handling:

- White space
- Instructions

Test program: Max

Max.asm

```
// Computes RAM[2] = max(RAM[0],RAM[1])

@R0
D=M           // D = RAM[0]
@R1
D=D-M         // D = RAM[0] - RAM[1]
@OUTPUT_RAM0
D;JGT         // if D>0 goto output R

// Output RAM[1]
@R1
D=M
@R2
M=D           // RAM[2] = RAM[1]
@END
0;JMP

(OUTPUT_RAM0)
@R0
D=M
@R2
M=D           // RAM[2] = RAM[0]

(END)
@END
0;JMP
```

with
symbols

MaxL.asm

```
// Symbol-less version

@0
D=M           // D = RAM[0]
@1
D=D-M         // D = RAM[0] - RAM[1]
@12
D;JGT         // if D>0 goto output RAM[0]

// Output RAM[1]
@1
D=M
@2
M=D           // RAM[2] = RAM[1]
@16
0;JMP

@0
D=M
@2
M=D           // RAM[2] = RAM[0]

@16
0;JMP
```

without
symbols

Test program: Rectangle

CPU Emulator (2.5) - /Users/admin/Desktop/nand2tetris/projects/06/rectangle/RectangleL.asm

File View Run Help

Slow Fast Animate: No animation View: Scr... Format: D...

ROM

0	@0
1	D=M
2	@16
3	M=D
4	@17
5	M=0
6	@16384
7	D=A
8	@18
9	M=D
10	@17
11	D=M
12	@16
13	D=D-M
14	@27
15	D;JGT
16	@18
17	A=M
18	M=-1
19	@17
20	M=M+1
21	@32
22	D=A
23	@18
24	M=D+M
25	@10
26	0;JMP
27	@27
28	0;JMP

RAM

0	40
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	20
17	12
18	16768
19	0
20	0
21	0
22	0
23	0
24	0
25	0
26	0
27	0
28	0

PC: 0 A: 0

F10

D: 0

ALU

D Input: 0

M/A Input: 0

ALU output: 0

Running...

Test program: Rectangle

Rectangle.asm

```
// Rectangle.asm
```

```
@R0
D=M
@n
M=D // n = RAM[0]

@i
M=0 // i = 0

@SCREEN
D=A
@address
M=D // base address of the Hack screen
```

```
(LOOP)
```

```
@i
D=M
@n
D=D-M
@END
D;JGT // if i>n goto END
...
```

with
symbols

RectangleL.asm

```
// Symbol-less version
```

```
@0
D=M
@16
M=D // n = RAM[0]
```

```
@17
M=0 // i = 0
```

```
@16384
D=A
@18
M=D // base address of the Hack screen
```

```
@17
D=M
@16
D=D-M
@27
D;JGT // if i>n goto END
...
```

without
symbols

Test program: Pong



Test program: Pong

Pong.asm

```
// Pong.asm
@256
D=A
@SP
M=D
@133
0;JMP
@R15
M=D
@SP
AM=M-1
D=M
A=A-1
D=M-D
M=0
@END_EQ
D;JNE
@SP
A=M-1
M=-1
(END_EQ)
@R15
A=M
0;JMP
@R15
M=D
...
```

Observations:

- Source code originally written in the Jack language
- The Hack code was generated by the Jack compiler and the Hack assembler
- The resulting code is 28,374 instructions long (includes the Jack OS)

Machine generated code:

- No white space
- “Strange” addresses
- “Strange” labels
- “Strange” pre-defined symbols

Nand to Tetris / www.nand2tetris.org / Chapter 6 / Copyright © Noam Nisan and Shimon Schocken

Testing options

Use your assembler to translate `Xxx.asm`,
generating the executable file `Xxx.hack`

Hardware simulator:

load `Xxx.hack` into the Hack Computer chip, then execute it

CPU Emulator:

load `Xxx.hack` into the supplied CPU emulator, then execute it

Assembler:

use the supplied Assembler to translate `Xxx.asm`;

Compare the resulting code to the binary code generated by *your* assembler.

Testing your assembler using the supplied assembler

Assembler (2.5) - /Users/admin/Dropbox (Slate Team)/hack/Coursera/week 6/program examples/Sum.hack

File Run Help

Source

```
// Computes RAM[1] = 1 + ... + RAM
@i
M=1 // i = 1
@sum
M=0 // sum = 0

(LLOOP)
@i // if i>RAM[0] goto STOP
D=M
@R0
D=D-M
@STOP
D;JGT
@i // sum += i
D=M
@sum
M=D+M
@i // i++
M=M+1
@LLOOP // goto LOOP
0;JMP

(STOP)
@sum
D=M
@R1
M=D // RAM[1] = the sum

(END)
@END
0;JMP
```

Destination

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000001000
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```

Comparison

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000001000
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```

Source Xxx.asm file

Xxx.hack file, translated by the supplied assembler

Xxx.hack file, translated by your assembler

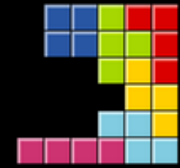
File compilation & comparison succeeded

Project 6 Resources

From NAND to Tetris

Building a Modern Computer From First Principles

www.nand2tetris.org



Home
Prerequisites
Syllabus

Course

Book
Software
Terms
Papers
Talks
Cool Stuff
About
Team
Q&A

Project 6: The Assembler

Background

Low-level machine programs are rarely written by humans. Typically, they are generated by compilers. Yet humans can inspect the translated code and learn important lessons about how to write their high-level programs better, in a way that avoids low-level pitfalls and exploits the underlying hardware better. One of the key players in this translation process is the *assembler* -- a program designed to translate code written in a symbolic machine language into code written in binary machine language.

This project marks an exciting landmark in our *Nand to Tetris* odyssey: it deals with building the first rung up the software hierarchy, which will eventually end up in the construction of a compiler for a Java-like high-level language. But, first things first.

Objective

Write an Assembler program that translates programs written in the symbolic Hack assembly language into binary code that can execute on the Hack hardware platform built in the previous projects.

Contract

There are three ways to describe the desired behavior of your assembler: (i) When loaded into `Prog.asm` file containing a valid Hack assembly language program should be translated into

All the necessary project 6 files are available in:
`nand2tetris / projects / 06`

Assembler: lecture plan

- ✓ The assembly process
- ✓ The Hack assembly language
- ✓ The assembly process: instructions
- ✓ The assembly process: symbols
- ✓ Developing an assembler
- ✓ Project 6 overview

References:

- <https://www.nand2tetris.org/course>
- https://www.youtube.com/watch?v=KBTg0ju4rxM&list=PLrDd_kMiAuNmIlp9vuPqCuttC1XL9VyVh

The C-instruction: symbolic and binary syntax

Symbolic syntax: *dest = comp ; jump*

Binary syntax: 1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

<i>comp</i>		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D A	D M	0	1	0	1	0	1
a=0	a=1						

<i>dest</i>	d1	d2	d3	effect: the value is stored in:
null	0	0	0	The value is not stored
M	0	0	1	RAM[A]
D	0	1	0	D register
MD	0	1	1	RAM[A] and D register
A	1	0	0	A register
AM	1	0	1	A register and RAM[A]
AD	1	1	0	A register and D register
AMD	1	1	1	A register, RAM[A], and D register

<i>jump</i>	j1	j2	j3	effect:
null	0	0	0	no jump
JGT	0	0	1	if out > 0 jump
JEQ	0	1	0	if out = 0 jump
JGE	0	1	1	if out ≥ 0 jump
JLT	1	0	0	if out < 0 jump
JNE	1	0	1	if out ≠ 0 jump
JLE	1	1	0	if out ≤ 0 jump
JMP	1	1	1	Unconditional jump



(see HDL Survival Guide @ www.nand2tetris.org)